

Christian_Herrera_HW8

March 24, 2026

1 Homework8 : Numpy and Random Variables (Questions)

2 Assignment Submission Guidelines

Please follow the guidelines below for submitting your assignment:

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM next Tuesday (03/24)**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: `Lastname_Firstname_AssignmentName`
- Example: `Alex_John_HW8.ipynb` and `Alex_John_HW8.pdf`.

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

3 Questions

3.0.1 Question 1: (10 Points)

Use NumPy to solve the following system of linear equations:

```
[1]: #3x+ 2y- z= 1  
  
#2x+ 2y+ 4z= -2  
  
#-x+ 1/2y- z= 0
```

```
[24]: import numpy as np
import random
import matplotlib.pyplot as plt

# The system of equations.
A = np.array([[3, 2, -1],
              [2, 2, 4],
              [-1, 1/2, -1]
              ])

# The solution.
B = np.array([1, -2, 0])

# Solve
solve = np.linalg.solve(A, B)

print(solve)
```

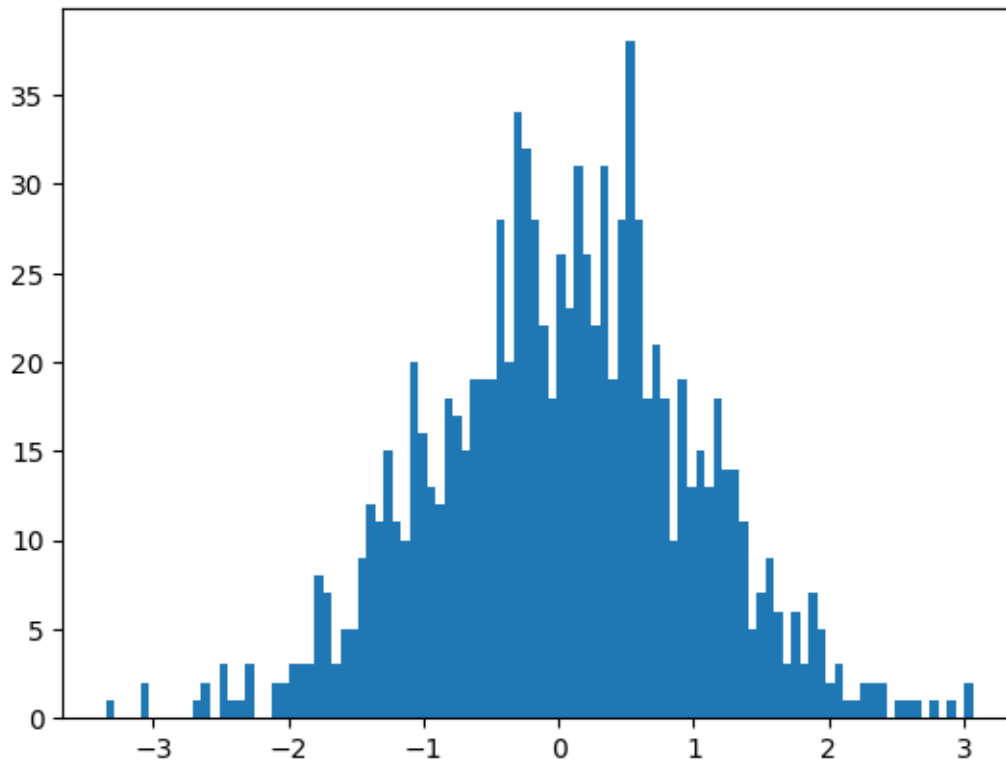
```
[ 0.36842105 -0.31578947 -0.52631579]
```

3.0.2 Question 2: (10 Points)

Generate an array of 1000 random numbers sampled from a standard normal distribution, then visualize it.

```
[3]: random_arr = np.random.randn(1000)

plt.hist(random_arr, bins=100)
plt.show()
```



3.0.3 Question 3: (10 Points)

Calculate the dot product of the following two vectors using NumPy:

```
[4]: A = [2, 3, 4]
      B = [5, 6, 7]
```

```
[5]: dot_prod = np.dot(A, B)

      print(dot_prod)
```

56

3.0.4 Question 4: (10 Points)

Random Numbers:: Generate a 1D array of 20 random integers between 10 and 50. Find out how many of them are even.

```
[43]: rand_arr = np.random.randint(low=10, high= 51, size=20)

even_list = list((i for i in rand_arr if i % 2 == 0))

print(f'This random generation between 10 and 50 has {len(even_list)} even_
↪number(s)')
```

This random generation between 10 and 50 has 13 even number(s)

3.0.5 Question 5: (10 Points)

Data Manipulation: Consider an array of dimension (8,8). Swap the first and last columns.

```
[28]: arr_8x8 = np.identity(8, dtype=int)

# Original col = other col
arr_8x8[:, [0, 7]] = arr_8x8[:, [7, 0]]

arr_8x8
```

```
[28]: array([[0, 0, 0, 0, 0, 0, 0, 1],
             [0, 1, 0, 0, 0, 0, 0, 0],
             [0, 0, 1, 0, 0, 0, 0, 0],
             [0, 0, 0, 1, 0, 0, 0, 0],
             [0, 0, 0, 0, 1, 0, 0, 0],
             [0, 0, 0, 0, 0, 1, 0, 0],
             [0, 0, 0, 0, 0, 0, 1, 0],
             [1, 0, 0, 0, 0, 0, 0, 0]])
```

3.0.6 Question 6: (10 Points)

Given a 2D NumPy array data, extract the following subsets:

- The first row.
- The last column.
- The subarray consisting of the first two rows and the last two columns.

```
[16]: arr = np.arange(1, 17).reshape(4, 4)

print(arr, '\n')

# First row
print(arr[0])
```

```
#Last column
print(arr[:,-1])

# The matrix of both
print(arr[:2,-2:])
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
[1 2 3 4]
[ 4  8 12 16]
[[3 4]
 [7 8]]
```

3.0.7 Question 7: (10 Points)

Statistics: Create a matrix of shape (5,5), find the mean and variance of the values in its 3rd column.

```
[40]: arr = np.arange(1, 26).reshape(5, 5) # matrix from a range

print(arr, '\n')

mean = arr[:, 2].mean()
variance = arr[:, 2].var()

print('The mean of column 3:')
print(mean)

print('The variance of column 3:')
print(variance)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
```

```
The mean of column 3:
13.0
The variance of column 3:
50.0
```

3.0.8 Question 8: (10 Points)

Linear Algebra: Given a 4x4 matrix, compute its transpose and then find the dot product of the matrix with its transpose.

```
[18]: matrix = np.array([[1, 2, 3, 4],
                        [5, 6, 7, 8],
                        [9, 10, 11, 12],
                        [13, 14, 15, 16]])
```

```
[42]: # The transpose
matrix_transpose = matrix.transpose()

dot_result = np.dot(matrix, matrix_transpose)

print(dot_result)
```

```
[[ 30  70 110 150]
 [ 70 174 278 382]
 [110 278 446 614]
 [150 382 614 846]]
```

3.0.9 Question 9: (10 Points)

Array Operations: Create two matrices, then Compute the element-wise multiplication of two matrices, then sum the values in each row of the resulting matrix.

```
[20]: matrix_1 = np.array([[1, 2, 3, 4],
                        [5, 6, 7, 8],
                        [9, 10, 11, 12],
                        [13, 14, 15, 16]])

matrix_2 = np.array([[1, 2, 3, 4],
                    [5, 6, 7, 8],
                    [9, 10, 11, 12],
                    [13, 14, 15, 16]])

matrix_mult = matrix_1 * matrix_2

print('Element-wise multiplication: \n',matrix_mult, '\n')
```

```
print('Row summation: \n',matrix_mult.sum(axis=1))
```

Element-wise multiplication:

```
[[ 1  4  9 16]
 [25 36 49 64]
 [81 100 121 144]
 [169 196 225 256]]
```

Row summation:

```
[ 30 174 446 846]
```

3.0.10 Question 10: (10 Points)

Advanced Array Creation: Create a 2D array of shape (7,7) where the border consists of ones and the inner part consists of zeros.

```
[21]: ones_arr = np.ones((7, 7), dtype=int) # Found it would print a decimal without
      ↪dtype specified

      # Slicing for the interior of the matrix and setting it to zero.
      ones_arr[1:6, 1:6] = 0

      print(ones_arr)
```

```
[[1 1 1 1 1 1 1]
 [1 0 0 0 0 0 1]
 [1 0 0 0 0 0 1]
 [1 0 0 0 0 0 1]
 [1 0 0 0 0 0 1]
 [1 0 0 0 0 0 1]
 [1 1 1 1 1 1 1]]
```

3.0.11 Question 11: (15 Points)

Create a function that asks the user for a numeric input to specify how many random integers to generate in a 1D array (between 1 and 100). Identify all prime numbers in the array and replace them with the number 1.

```
[23]: def is_prime(n):
    # Prime?
    if n < 2:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True

def replace_primes_with_one():
    # Replace primes with one.
    size = int(input("How many random integers do you want? "))

    arr = np.random.randint(1, 101, size)

    print("Original array:")
    print(arr)

    # All of them.
    for i in range(len(arr)):
        if is_prime(arr[i]):
            arr[i] = 1

    # Here you go.
    print("Updated array:")
    print(arr)

replace_primes_with_one()
```

Original array:

[95 48 99 47 64 99 86 83 91 47]

Updated array:

[95 48 99 1 64 99 86 1 91 1]

4 Question 12: Number Guessing Game (25 Points)

4.0.1 Objective

The objective of this homework assignment is to create a Number Guessing game using Python. This will help you practice and reinforce your understanding of basic programming concepts such as conditionals, loops, functions, user input, and random number generation.

4.0.2 Instructions

1. Setup the Project

- Create a new Python file named `number_guessing_game.py`.

2. Game Requirements

- The game should randomly select a number within a specified range.
- The user should be prompted to guess the number.
- The game should provide feedback if the guess is too high, too low, or correct.
- The game should continue until the user guesses the correct number or decides to stop playing.

3. Detailed Steps to Implement

1. Import the Necessary Module

- Use the `random` module to generate a random number. “python import random

2. Generate the Random Number

- Define a function `generate_random_number(min_value, max_value)` to generate a random number within the given range.

3. Get User Guess

- Define a function `get_user_guess()` to prompt the user to enter their guess and validate the input.

4. Check Guess

- Define a function `check_guess(guess, target)` to check if the guess is too high, too low, or correct.

5. Main Game Function

- Define the main game function `play_game()` to implement the game loop and manage the game logic.

```
[ ]: # The output below is from me testing the code.
```

```
Welcome to the Number Guessing Game!
I'm thinking of a number between 1 and 100.
Too high. Try again.
Too low. Try again.
Too low. Try again.
Too low. Try again.
Too low. Try again.
Too low. Try again.
Too low. Try again.
Too high. Try again.
Correct! You guessed the number.
```

```
[ ]:
```